



Embedded Systems Course - Interrupts and Exception handling

Siddhesh Raut, Prakash Balasubramanian

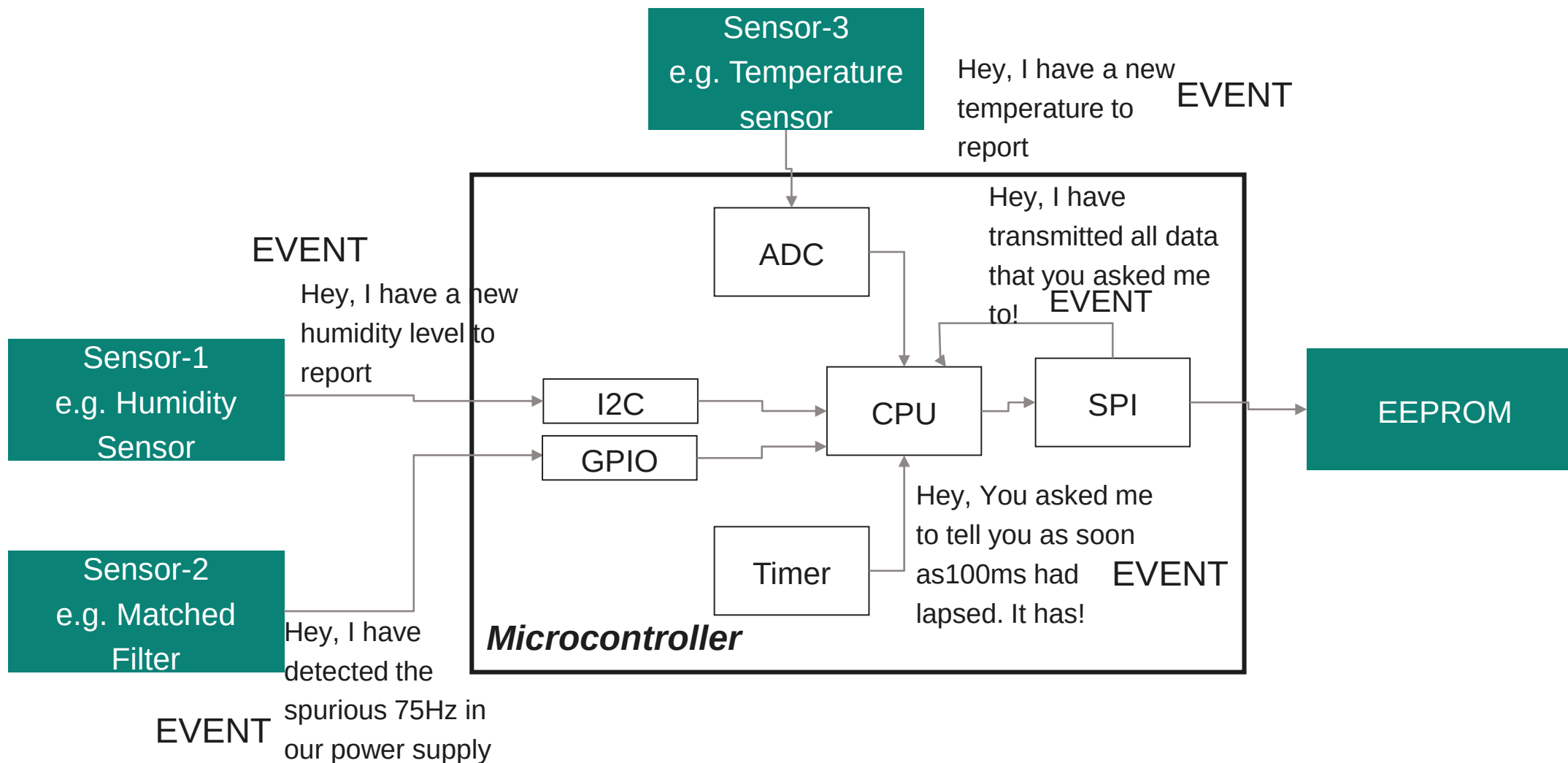
15 Nov 2025



Outline

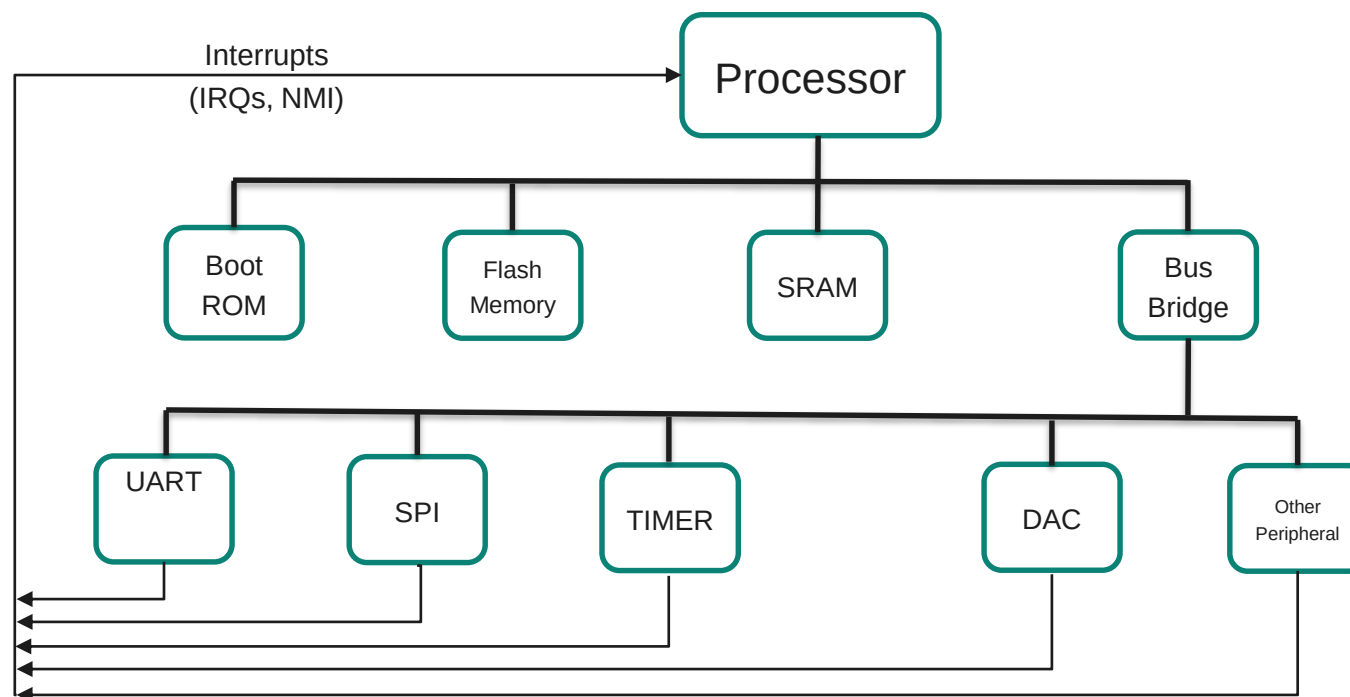
- Event Driven System.
- Polling Method of event.
- Exception, Interrupt and Exception handler.
- Different types of Exceptions
- Operation Modes
- Register Bank
- Interrupt Controller
- Brief overview of NVIC
- NVIC Registers
- Vector Table
- Exception Entry Sequence
- Exception Exit Sequence
- Interrupt Process(complete)

Event driven embedded system – 1/2

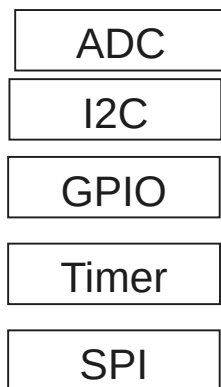


Event driven embedded system – 2/2

- Event represents significant occurrence or state changes within a system
 - It represented as signals that convey information about particular occurrence
- Event are asynchronous in nature
- Events can occur at any time!



Methods of handling events – 1/2



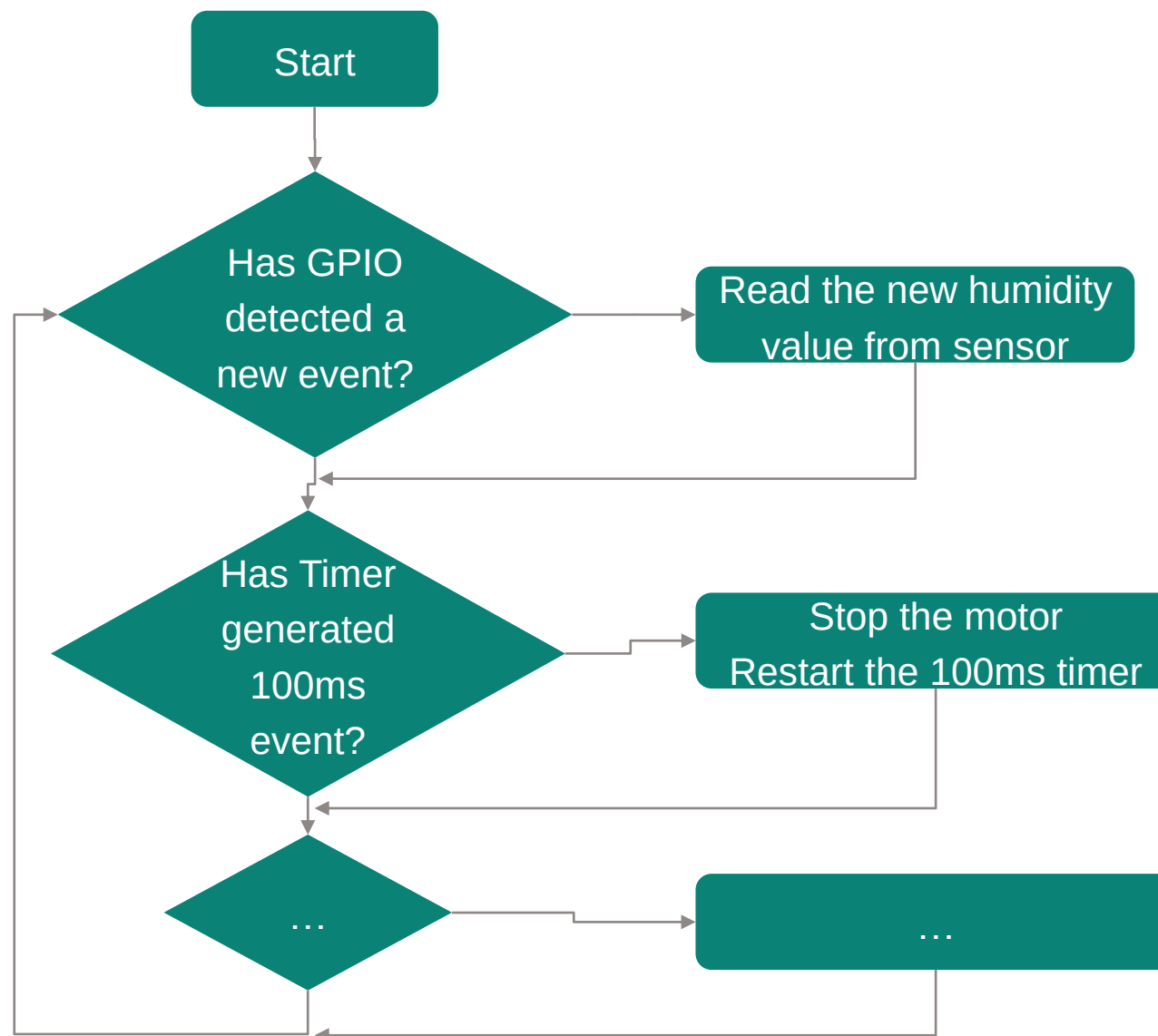
Polling mode

What should be the correct order of polling?

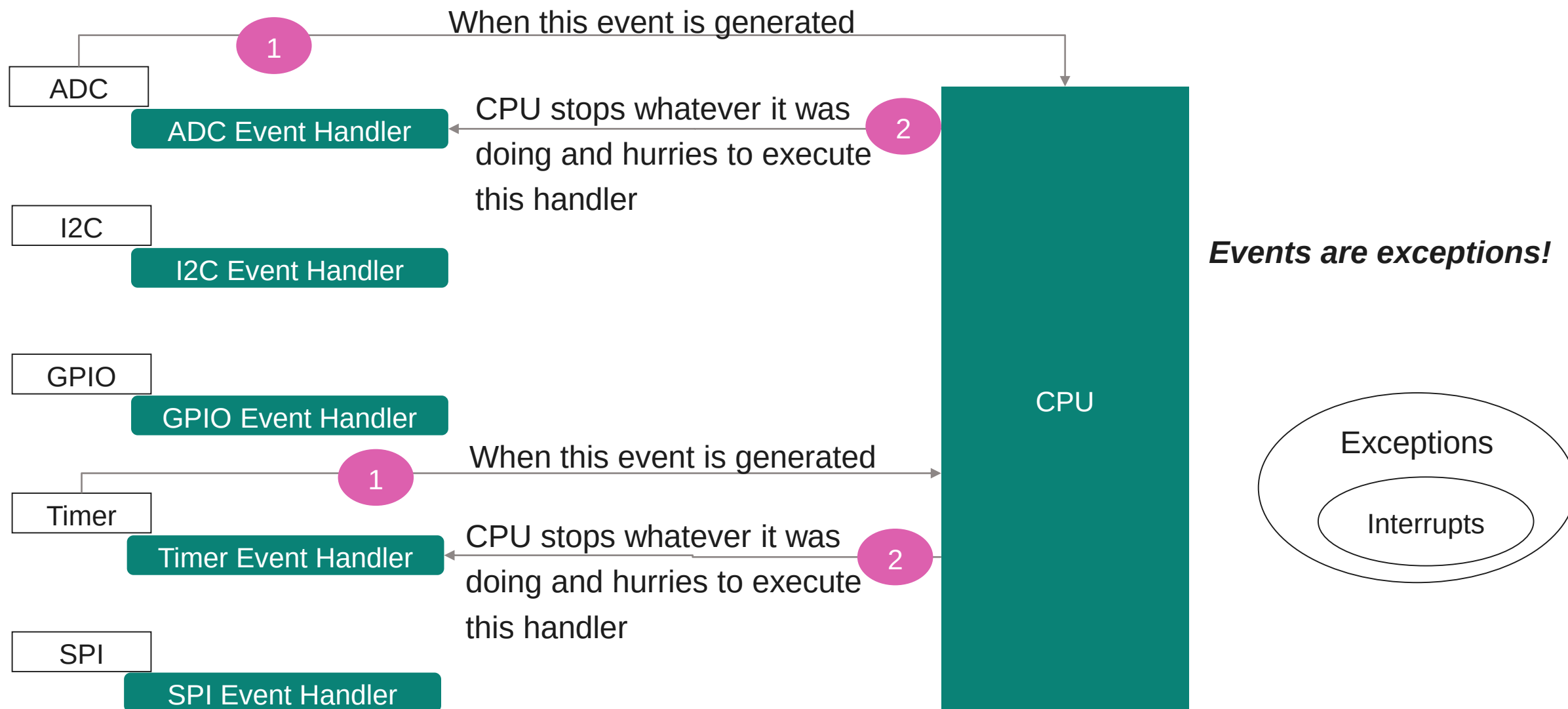
Can the event be lost by the time we get to a peripheral?

Energy inefficient. CPU is always burning power

Use polling mode as necessary. It is NOT criminal to do so!



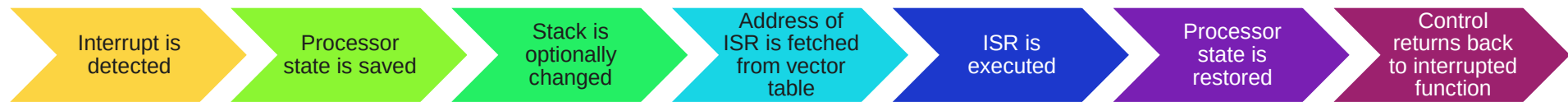
Methods of handling events – 2/2



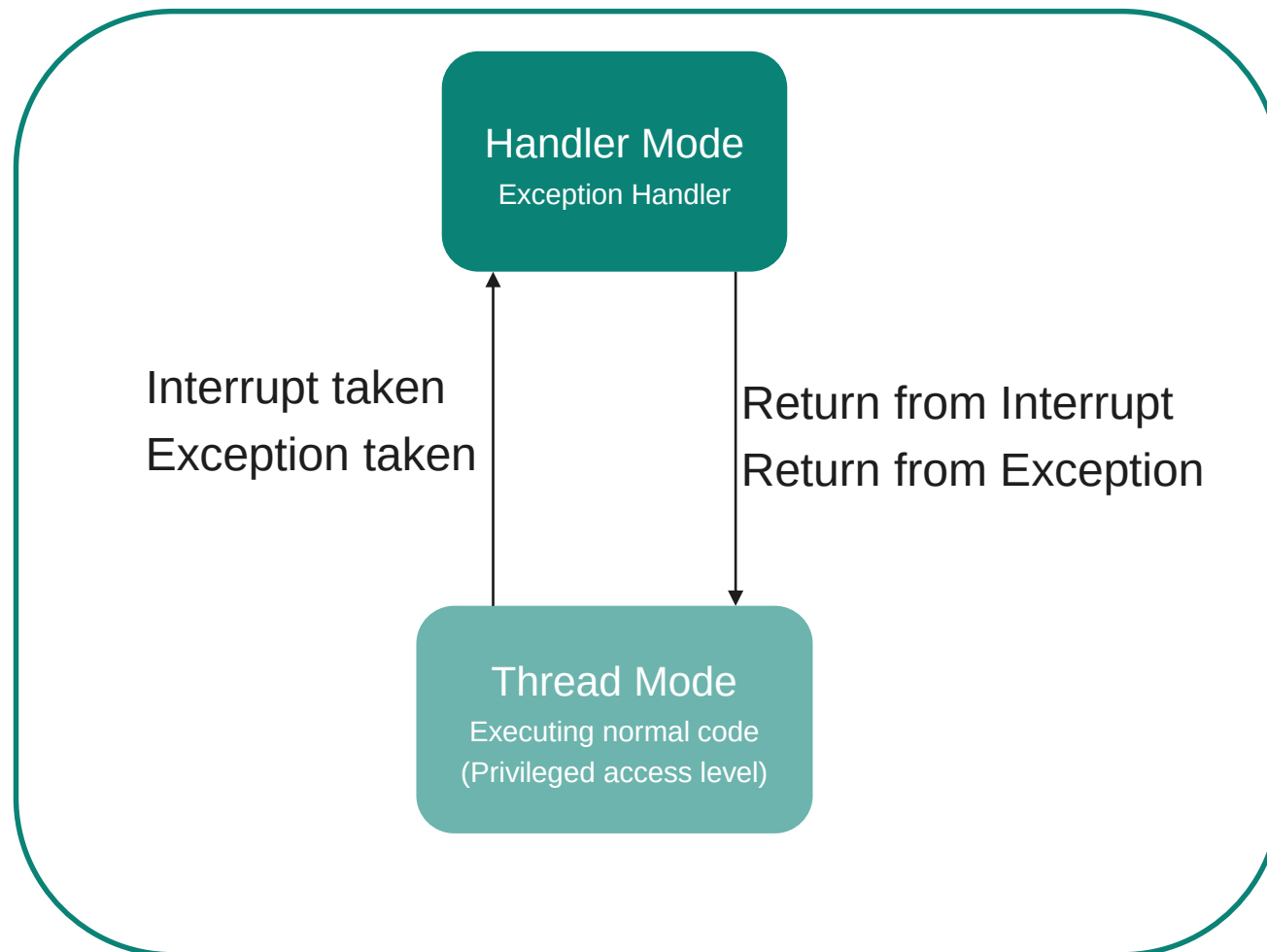
Exception, Interrupt and Exception handler

- Any condition that needs to halt the normal sequential execution of instructions.
- In Exception the device notices the CPU that it requires its attention.
- An exception is not a protocol, its a hardware mechanism.
- Interrupts are a subset of exceptions. External to processor core.
- when an exception occurred, instead of continuing program execution, the processor suspends the current executing task and executes a part of the program code called the *exception handler*.
- If the exception handler is associated with an interrupt event, then it can also be called as interrupt handler, or Interrupt Service Routine (ISR).

Overview of interrupt handling



Operation Modes and States



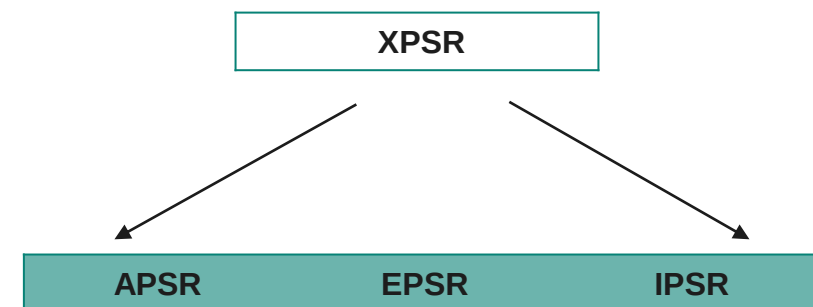
Processor state - Register Bank and special registers

Register Bank

R0
R1
R2
R3
R4
R5
R6
R7
R8
R9
R10
R11
R12
R13(BANKED)->STACK POINTER
R14-> LINK REGISTER
R15-> PROGRAM COUNTER



Special registers



PRIMASK(Interrupt Mask Register)

CONTROL(Stack definition)

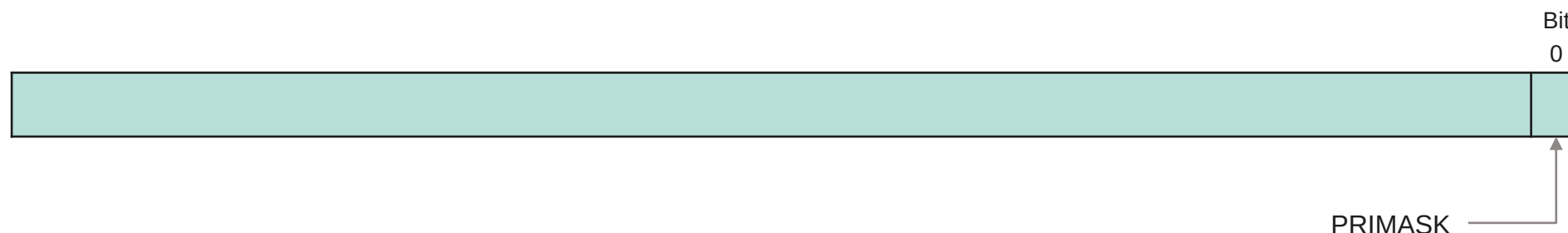
Stack management by CONTROL register

- It is used for accessing the stack memory via PUSH and POP operations..
- There are physically two different stack pointers:
 - The Main Stack Pointer (MSP, or SP_main in ARM documentation) is the default Stack Pointer after reset and is used when running exception handlers.
 - The Process Stack Pointer (PSP, or SP_process in ARM documentation) can only be used in Thread mode (when not handling exceptions).
- The stack pointer selection is determined by the CONTROL register, one of the special registers
 - SPSEL -1 (PSP)
 - SPSEL-0 (MSP)



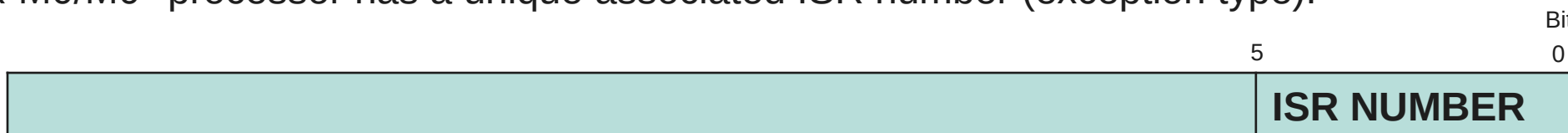
CPU Level interrupt management - PRIMASK and IPSR

- The PRIMASK register is a 1-bit wide interrupt mask register. When set, it blocks all interrupts apart from the Non-Maskable Interrupt (NMI) and the HardFault exception.



PRIMASK Register

- The IPSR contains the current executing ISR (Interrupt Service Routine) number. Each exception on the Cortex-M0/M0⁺ processor has a unique associated ISR number (exception type).

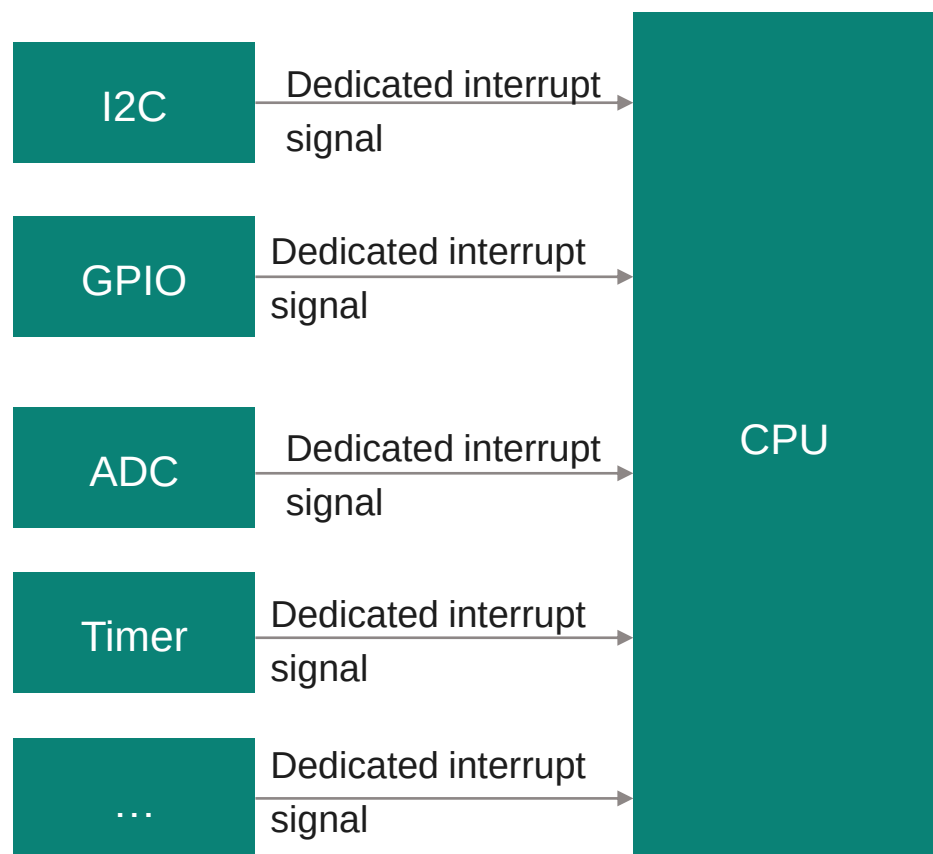


IPSR Register

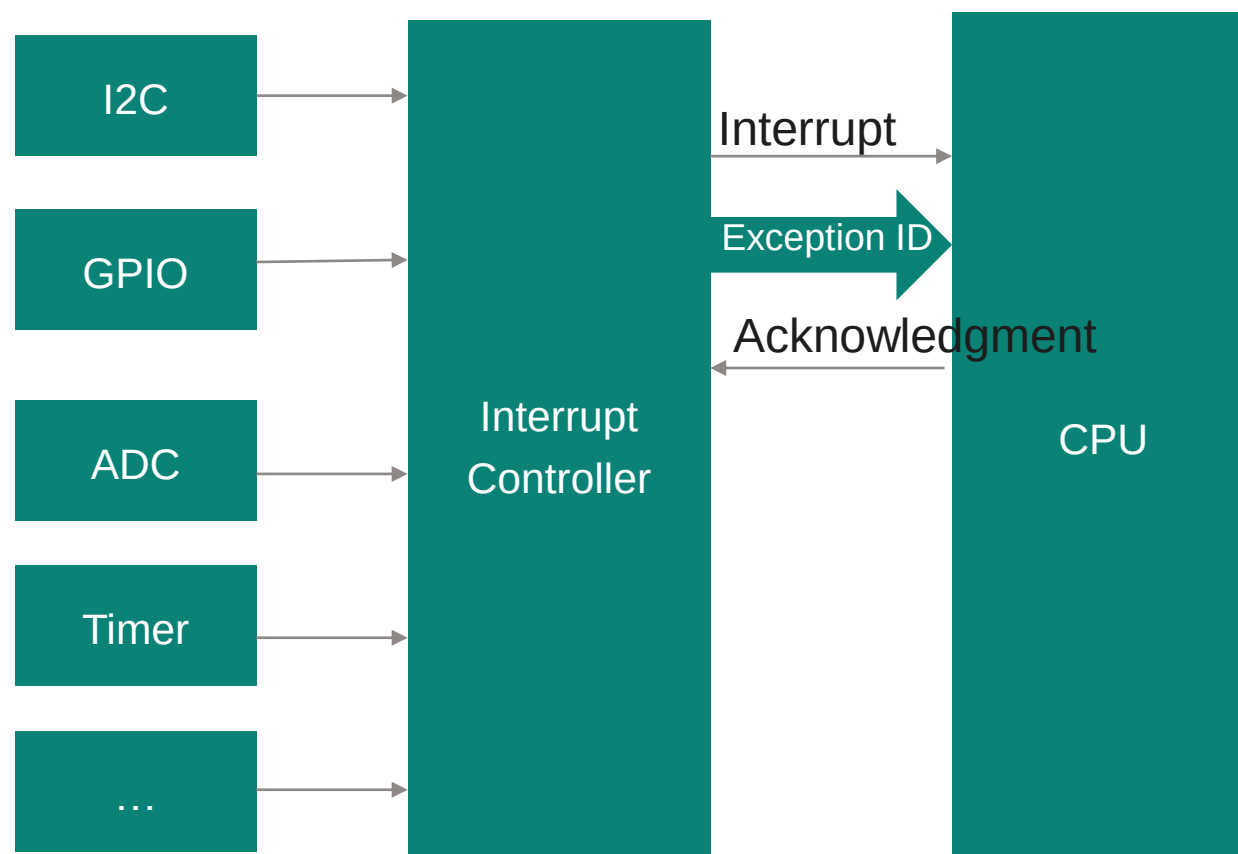
Different types of Exceptions

- There are seven types of exception:
 - **Non-Maskable Interrupt:** The NMI is like IRQ, but it cannot be disabled and has the highest priority apart from the reset. It is very useful for safety critical systems like industrial control or automotive.
 - **HardFault:** HardFault is an exception type dedicated for handling fault conditions during program execution
 - **SVCall (Supervisor Call):** SVCall exception takes place when the SVC instruction is executed. SVC is usually used in system with Operating System (OS), allowing applications to access to system services.
 - **Pending Service Call:** Pending Service Call (PendSV) is another exception for applications with OS. But PendSV can be delayed
 - **System Tick Timer:** The SysTick Timer inside the NVIC is another feature for OS application. Almost all OS need a timer to generate periodic interrupt for system maintenance works like context switching etc.
 - **Interrupt**

How to manage several interrupts?



Too many signals directly landing on CPU
Never used in practice

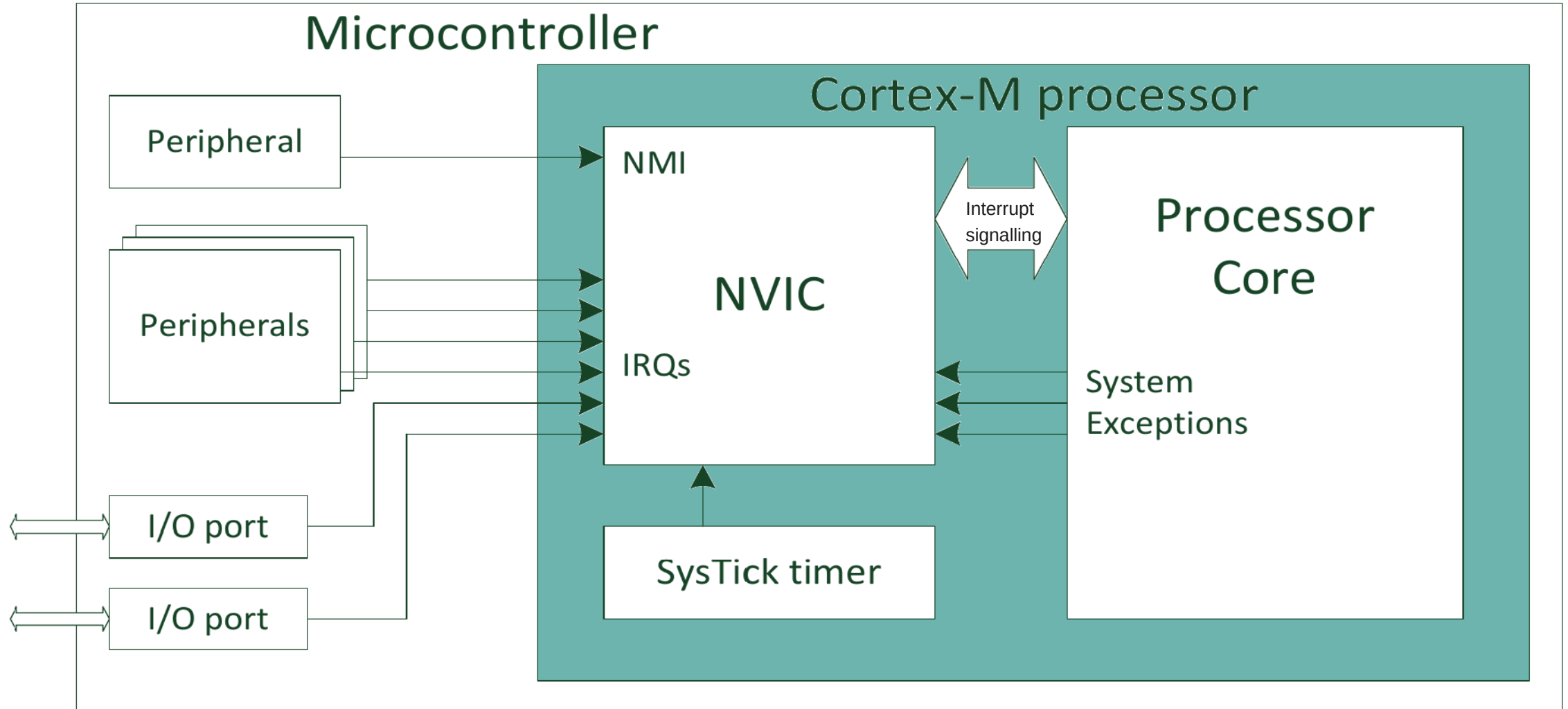


Enable/Disable individual interrupts
Assign priorities to interrupts
Maintains pending interrupt status – No interrupt event is lost

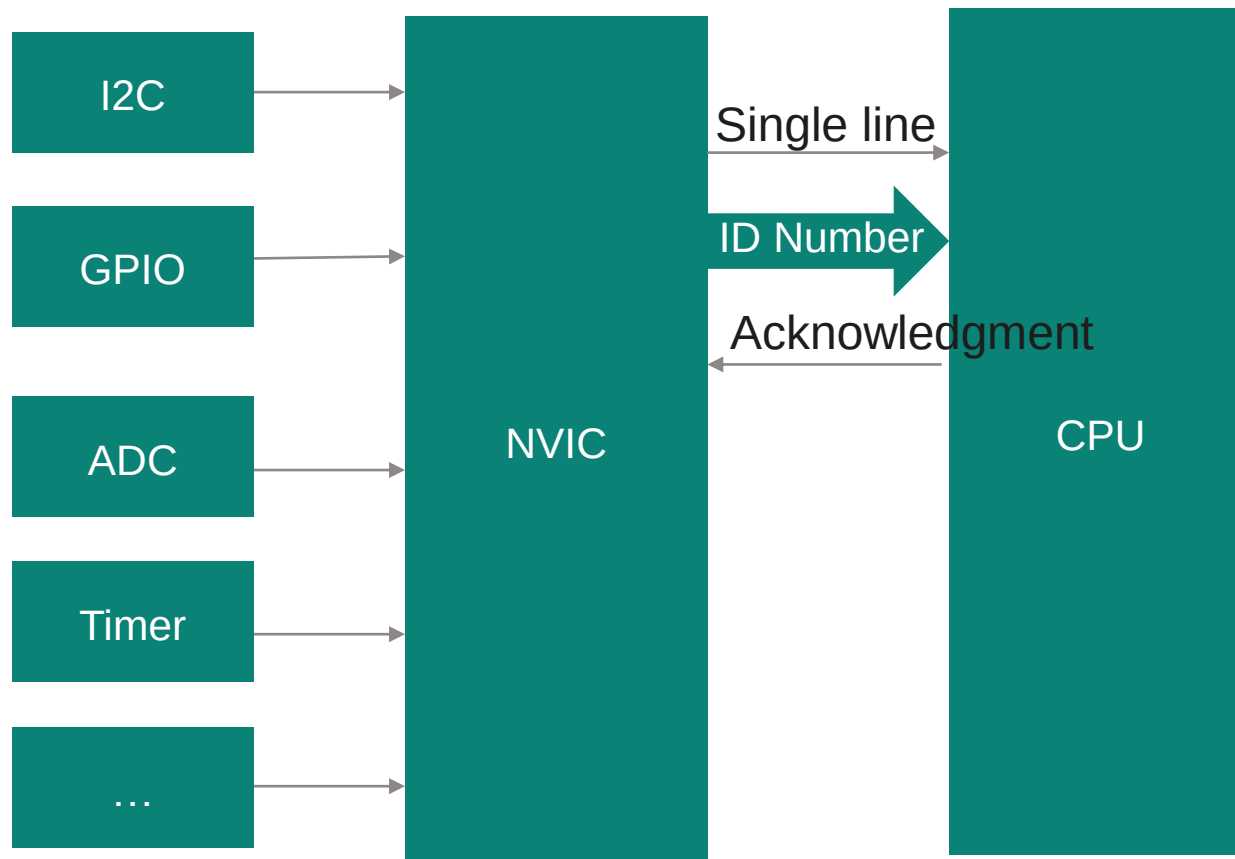
Interrupt Controller on Cortex M0+ : NVIC

- The NVIC is a programmable unit that allows software to manage interrupts and exceptions.
- It has a number of memory mapped registers for the following:
 - Enabling or disabling of each of the interrupts
 - Defining the priority levels of each interrupts and some of the system exceptions
 - Enabling the software to access the pending status of each interrupt, including the capability to trigger interrupts by setting pending status in software.

Interrupt Controller



How to enable/disable interrupts?



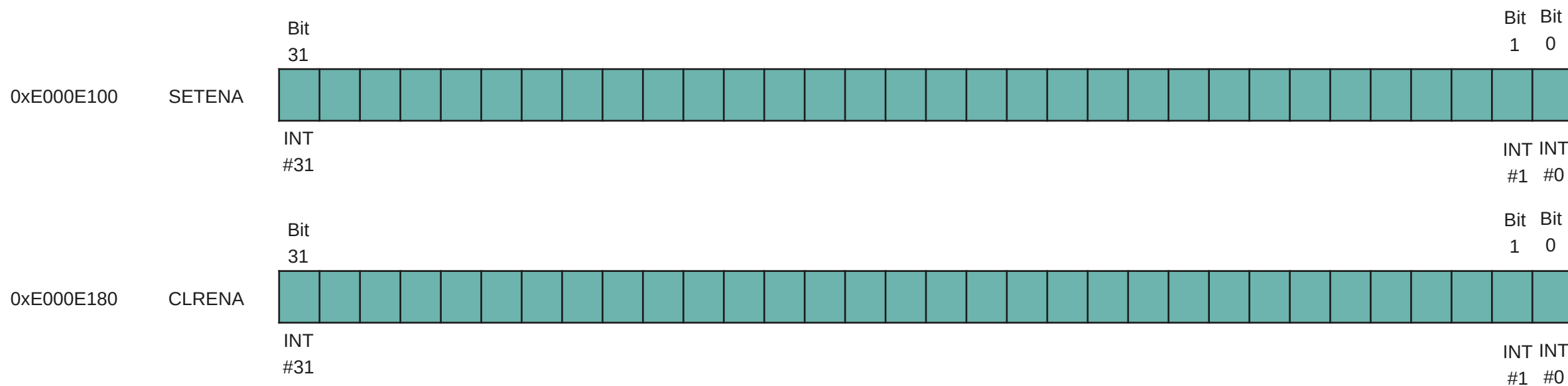
Peripherals generate events.
Enable them first

Enable interrupt generation
for desired peripheral events

Enable interrupt detection at
CPU level

NVIC Registers -> Interrupt Enable and Clear Enable

- Used to control the enable/disable of the IRQs (exception 16 and above).
- Width of this register is 32 bit and minimum size is 1 bit.
- To enable an interrupt, the SETENA address is used, and to disable an interrupt, the CLRENA address is used.

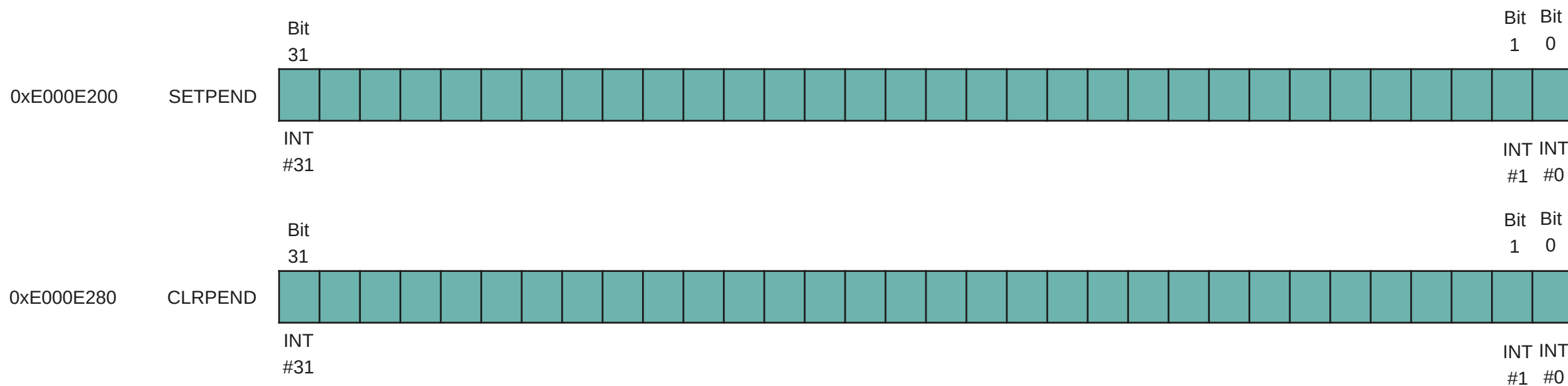


NVIC Registers -> Interrupt Enable and Clear Enable

- /* Enable interrupt #2 */
 - *((volatile unsigned long *) (0xE000E100)) = 0x4;
- /* Disable interrupt #2 */
 - *((volatile unsigned long *) (0xE000E180)) = 0x4;
- By using CMSIS Function
- /* Enable Interrupt IRQn value of 0 refer to Interrupt #0 */
 - void NVIC_EnableIRQ(IRQn_Type IRQn);
- /* Disable Interrupt e IRQn value of 0 refer to Interrupt #0 */
 - void NVIC_DisableIRQ(IRQn_Type IRQn);

NVIC Registers -> Interrupt Pending Set and Clear Register

- Interrupt takes place but cannot be processed immediately, for example, if the processor is serving another higher priority interrupt, the IRQ will be pended
- The interrupt pending status can be accessed, or modified, through the Interrupt Set Pending (SETPEND) and Interrupt Clear Pending (CLRPEND) register addresses



Interrupt Pending Set and Clear Register

- /* Enable interrupt #2 */
 - *((volatile unsigned long *) (0xE000E100)) = 0x4;
- /* Pend interrupt #2 */
 - *((volatile unsigned long *) (0xE000E200)) = 0x4;
- /* Clear interrupt #2 pending status */
 - *((volatile unsigned long *) (0xE000E280)) = 0x4;
- By using a library function (e.g. CMSIS Library)
- /* Set pending status of a interrupt */
 - void NVIC_SetPendingIRQ(IRQn_Type IRQn);
- /* Clear pending status of a interrupt */
 - void NVIC_ClearPendingIRQ(IRQn_Type IRQn);
- /* Return true if the interrupt pending status is 1 */
 - uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn);

NVIC Registers -> Interrupt Priority Level

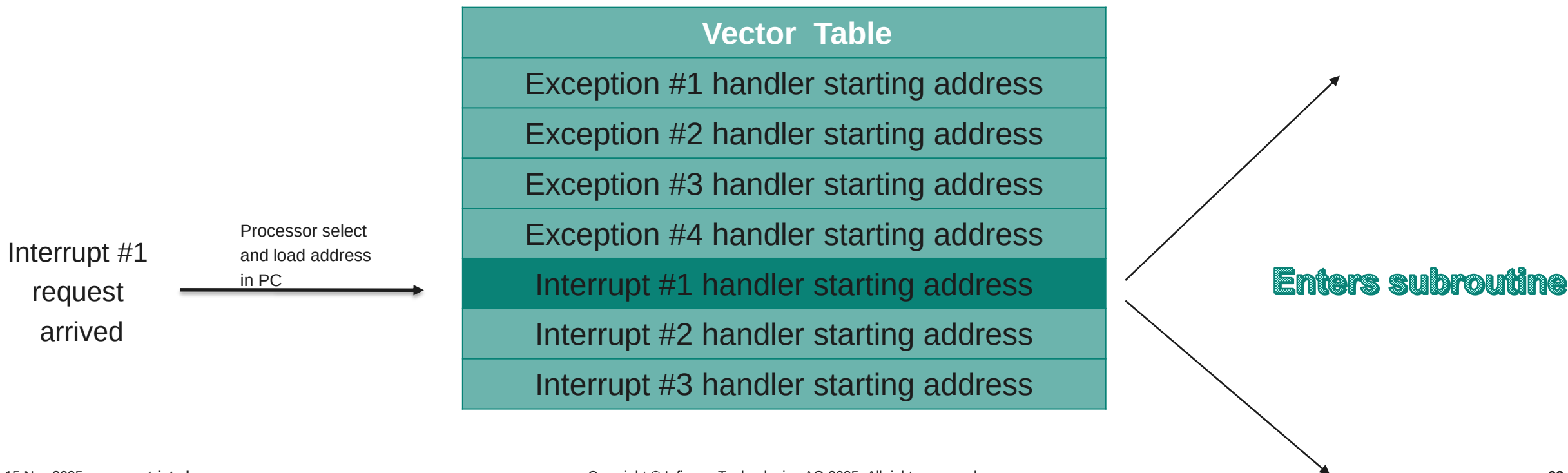
- Each external interrupt has an associated priority level register. Each of them is 2 bit wide, occupying the two MSBs of the Interrupt Priority Level Registers.

Bit	31 30	24 23 22	16 15 14	8 7 6	0				
0xE000E41C	31		30		29		28		IPR7
0xE000E418	27		26		25		24		IPR6
0xE000E414	23		22		21		20		IPR5
0xE000E410	19		18		17		16		IPR4
0xE000E40C	15		14		13		12		IPR3
0xE000E408	11		10		9		8		IPR2
0xE000E404	7		6		5		4		IPR1
0xE000E400	IRQ 3		IRQ 2		IRQ 1		IRQ 0		IPR0

- By using CMSIS function
- // Set the priority level of an interrupt or a system exception
- void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority);
- // return the priority level of an interrupt or a system exception
- uint32_t NVIC_GetPriority(IRQn_Type IRQn);

Vector Table

- The interrupt handling in the Cortex-M Processor is vectored, which means the processor's hardware automatically determines which interrupt or exception to service.
- After receiving an IRQ of exception event, it will need to know the starting address of the handler, and the vector table is a lookup table in the memory that provides such information.



Vector Table(conti.)

Memory Address		Exception Number
	⋮	
0x0000004C	Interrupt#3 vector	19
0x00000048	Interrupt#2 vector	18
0x00000044	Interrupt#1 vector	17
0x00000040	Interrupt#0 vector	16
0x0000003C	SysTick vector	15
0x00000038	PendSV vector	14
0x00000034	Not used	13
0x00000030	Not used	12
0x0000002C	SVC vector	11
0x00000028	Not used	10
0x00000024	Not used	9
0x00000020	Not used	8
0x0000001C	Not used	7
0x00000018	Not used	6
0x00000014	Not used	5
0x00000010	Not used	4
0x0000000C	HardFault vector	3
0x00000008	NMI vector	2
0x00000004	Reset vector	1
0x00000000	MSP initial value	0

Note : LSB of each vector must be set to 1 to indicate Thumb state

Exception Entry Sequence

- When an exception takes place, a number of things happen as follows:

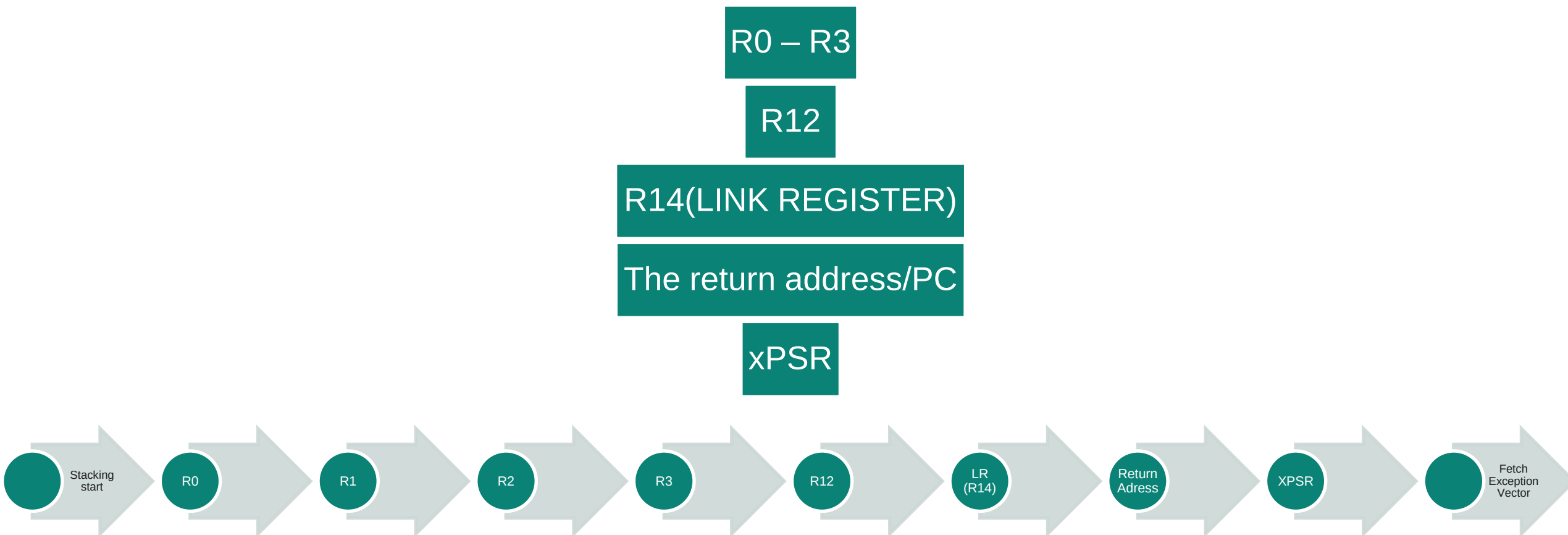
Stacking and update of one of the Stack Pointers (SPs)

Vector fetch (determine starting address of ISR) and update R15 (PC)

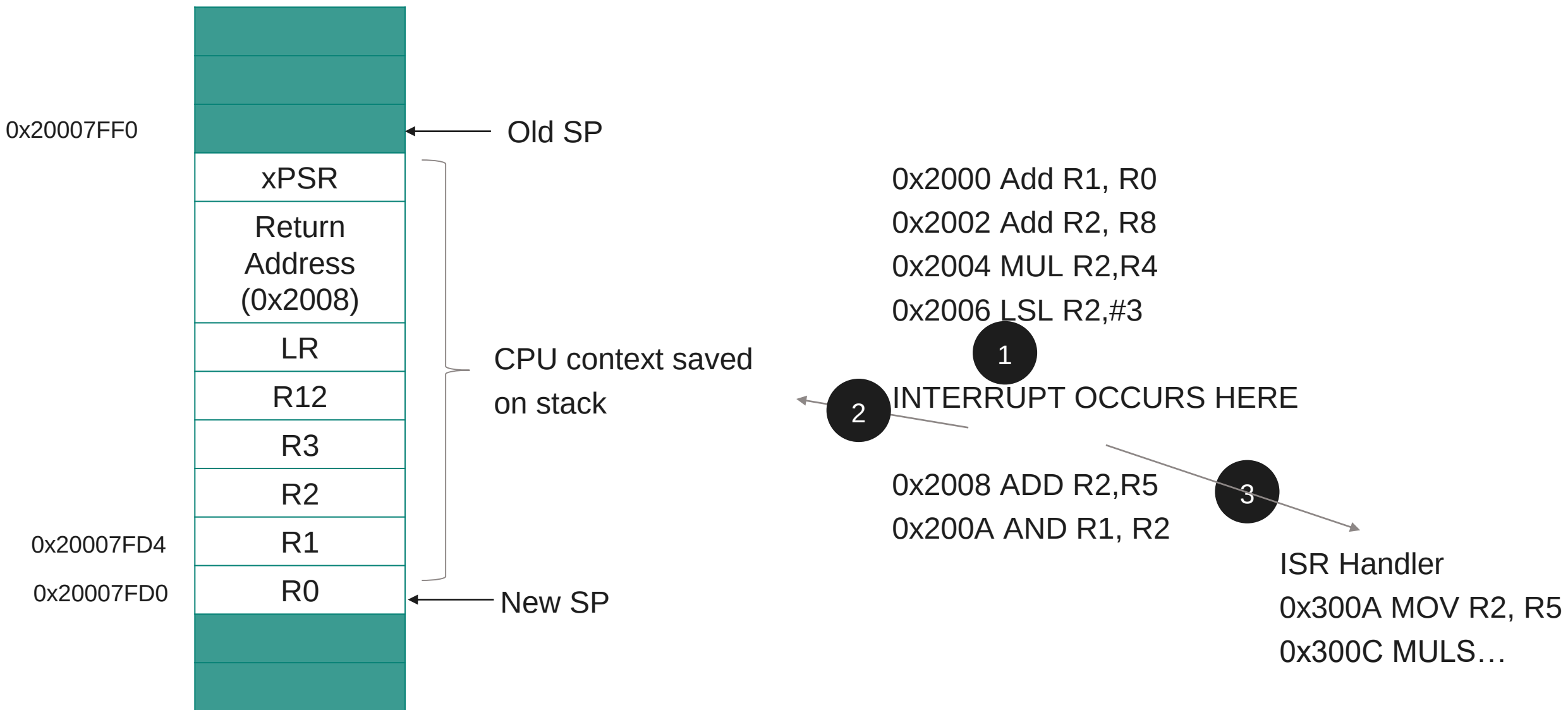
Registers update (LR, Internal Program Status Register (IPSR), NVIC registers)

Stacking – 1/2

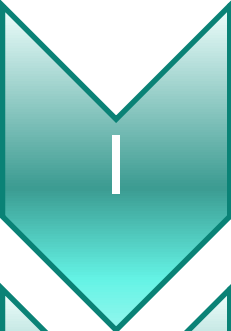
- When an exception takes place, eight registers are pushed:

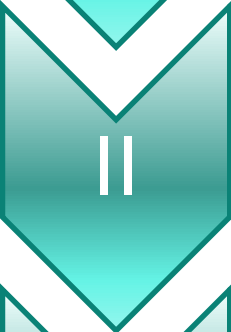


Stacking – 2/2



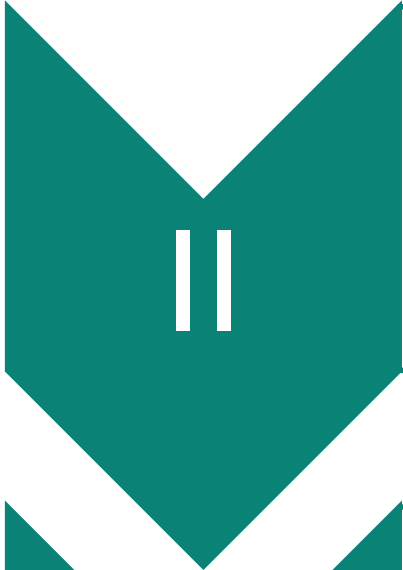
Vector Fetch and Update PC

- 
- After the stacking, the processor then fetches the exception vector (starting address of the ISR) from the vector table

- 
- Vector updated to the PC.

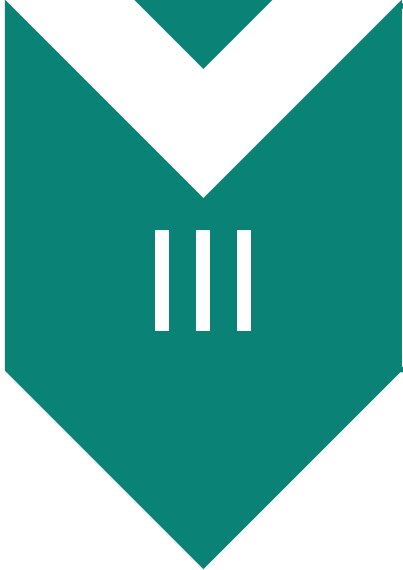
- 
- Instruction fetch of the exception handler execution starts from this address.

Additional Registers Updates



II

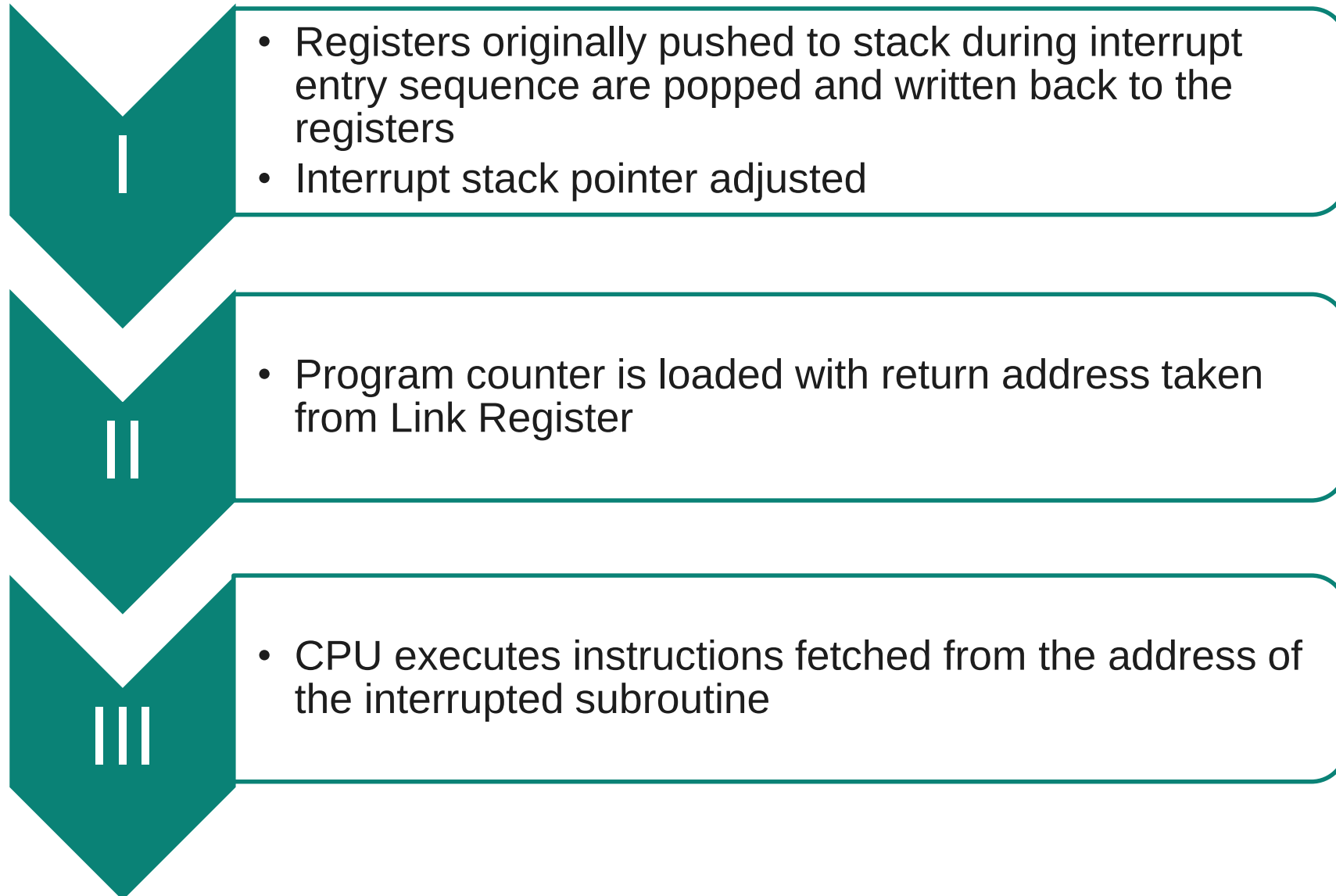
- the IPSR is also updated to the exception number of current serving exception.



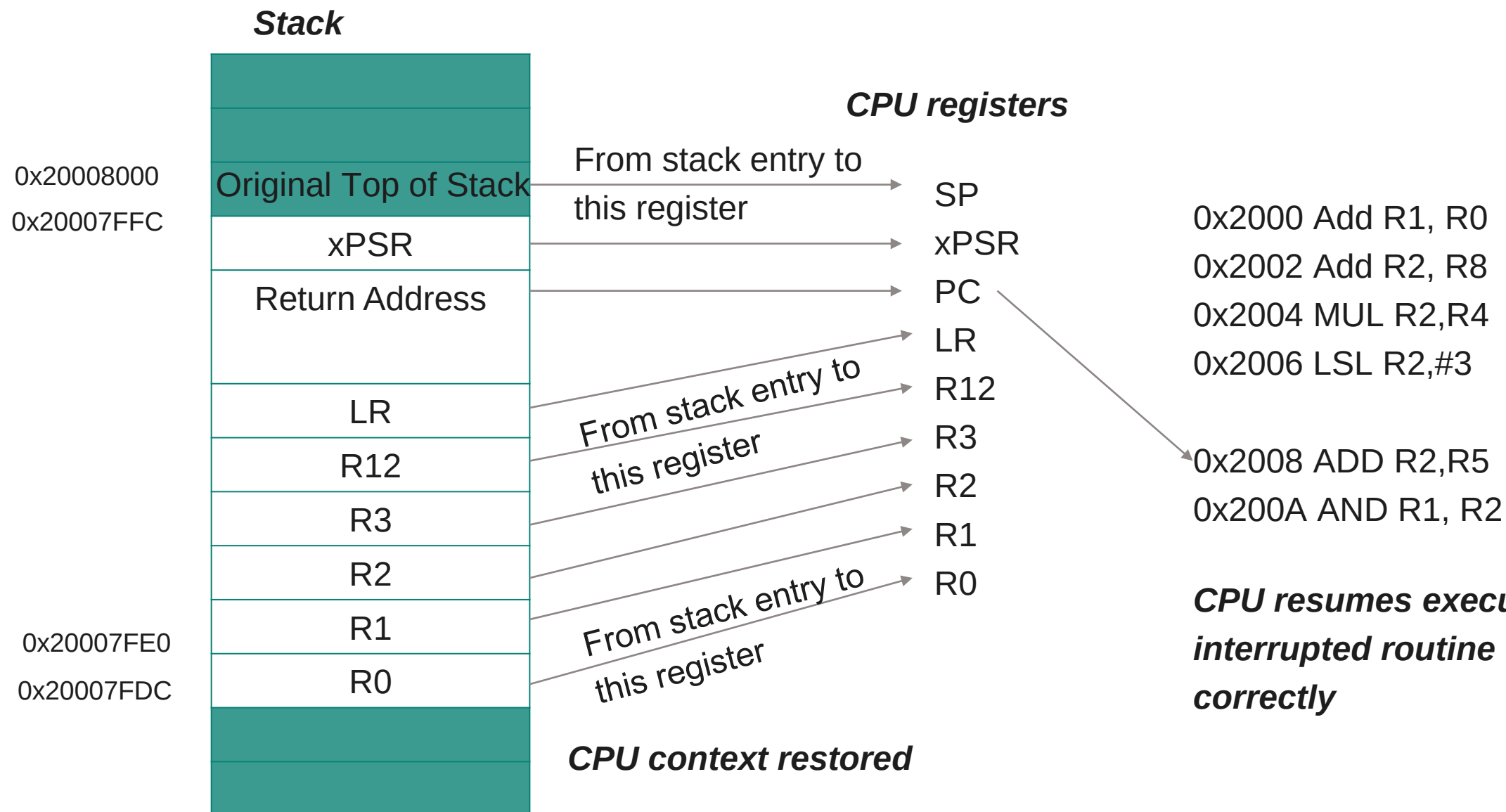
III

- In addition, a few NVIC registers get updated

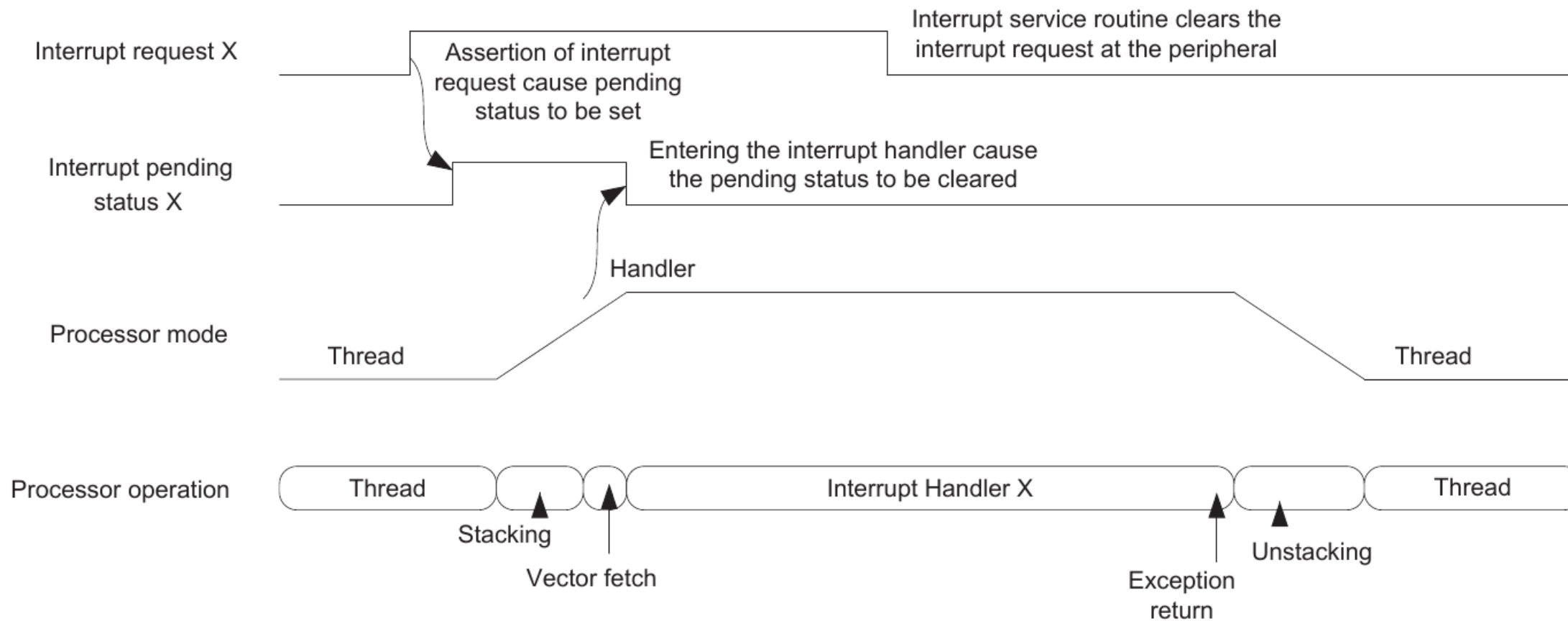
Returning back to the interrupted subroutine



Returning back from interrupt



Interrupt Process



Key Takeaway-1

- Interrupts can occur at any time
- When an interrupt occurs (and assuming it has been enabled), the CPU does the following (in the order listed below)
 - Stores the context of the thread being interrupted
 - The context is R0-R3, R12, LR, Return Address, xPSR
 - Updates the Stack Pointer by deducting 32 from its old value
 - Indexes into the Interrupt Vector Table, retrieves the start address of the ISR and programs the PC with that address
 - With this, the CPU is now ready to execute the ISR
 - The last few instructions of the ISR are to pop the saved R0-R3, R12, LR, Return Address, xPSR out of the stack and restore those data back into the registers
 - The CPU now is able to resume the interrupted program

Key Takeaway-2a

- Setting interrupt priorities
- Suppose your system only has to deal with IRQ-5, IRQ-8, IRQ-9, IRQ-13 and IRQ-14
- You would like to set their priorities to 2, 1, 3, 3 and 0 respectively
- First identify which of the NVIC IPR registers are applicable
 - IPR1 (0xE000E404) for IRQ-5
 - IPR2 (0xE000E408) for IRQ-8 and IRQ-9
 - IPR-3 (0xE000E40C) for IRQ-13 and IRQ-14
- Next identify the bit positions of the required interrupts in their IPR registers
 - IRQ-5 priority bits are at 15:14
 - IRQ-8 bits are at 7:6, IRQ-9 bits are at 15:14
 - IRQ-13 bits are at 15:14, IRQ-14 bits are at 23:22

Key Takeaway-2b

- Setting priority of IRQ-5 to 2 by Programming IPR1
 - `*( (uint32_t *)0xE000E404) = 2U << 14;`
- Setting priority of IRQ-8 to 1 and IRQ-9 to 3 by Programming IPR2
 - `*( (uint32_t *)0xE000E408) = (1U << 6) | (3U << 14);`
- Setting priority of IRQ-14 to 3 and IRQ-15 to 0 by Programming IPR2
 - `*( (uint32_t *)0xE000E408) = (3U << 22) | (0U << 30);`

Enabling and disabling interrupts on NVIC

- Program SETENA to enable interrupts.
- Program CLRENA to disable interrupts
- If you want to enable IRQ-5, IRQ-8, IRQ-9, IRQ-14 and IRQ-15
 - Write Bit 1 into bit positions 5, 8, 9, 14 and 15 respectively
 - `*( (uint32_t *)0xE000E100) = (1U << 5) | (1U << 8) | (1U << 9) | (1U << 14) | (1U << 15);`
- If you want to disable IRQ-15
 - Write Bit 1 into Bit position 15 of CLRENA
 - `*( (uint32_t *)0xE000E180) = (1U << 15);`
- If you want to know which of the interrupts are enabled, read either SETENA or CLRENA.
 - Both will return a bit mask containing 1s. Those bit-positions containing a 1 indicate an enabled interrupt.
 - `uint_32 ints_status = *( (uint32_t *)0xE000E100);`

Is any interrupt pending?

- Suppose you want to know if any of the interrupts are pending, then you may read either SETPEND or CLRPEND register
- They will return a bit-mask of pending interrupts
- `uint32_s ints_pend_status = *( (uint32_t *)0xE000E200);`
- Suppose you determine IRQ-5 is pending, but would like to disregard it (for some reason), then force clear its pending status by writing to CLRPEND register
- `*( (uint32_t *)0xE000E200) = (1U << 5);`
- Suppose you want to forcibly generate IRQ-8, write a 1 in Bit position 8 of SETPEND register
- `*( (uint32_t *)0xE000E100) = (1U << 8);`

Which interrupt is currently active

- On the PSOC, there are a total of 44 interrupt entries (16 for Exceptions and 28 for interrupts)
- 44 interrupts require 6 bits for encoding
- So read the 6 LSBs of IPSR register
- Here is what you should expect
 - When CPU is executing your regular program, $\text{IPSR}[5:0] = 0$
 - When CPU is executing Hard Fault exception, $\text{IPSR}[5:0] = 2$
 - When CPU is executing IRQ-0, $\text{IPSR}[5:0] = 16$
 - When CPU is executing IRQ-1, $\text{IPSR}[5:0] = 17$
 - When CPU is executing IRQ-2, $\text{IPSR}[5:0] = 18$
 - When CPU is executing IRQ-3, $\text{IPSR}[5:0] = 19$
 - And so on

